

PATERNI PONAŠANJA

Strategy patern

Strategy patern se koristi kada za isti problem postoji više algoritama, a klijent bira koji algoritam želi koristiti. U eVrtić sistemu ovaj patern se može primijeniti kod generisanja sedmičnog i mjesečnog sažetka aktivnosti. Oba sažetka koriste dnevne izvještaje, ali se razlikuju po periodu obrade, načinu agregacije i količini podataka. Umjesto da SazetakService sadrži više uslovnih grananja, uvodi se interfejs ISazetakStrategija, a konkretne klase SedmicnaSazetakStrategija i MjesecnaSazetakStrategija implementiraju vlastiti algoritam generisanja.

Prednost ovog pristupa je u tome što se kasnije može dodati nova strategija, na primjer GodisnjaSazetakStrategija ili DetaljniSazetakStrategija, bez izmjene postojećeg servisa i bez narušavanja postojećih model klasa.

State patern

State patern omogućava promjenu ponašanja objekta u zavisnosti od njegovog trenutnog stanja. Pogodan je kada objekat prolazi kroz više jasno definisanih statusa i kada se za svaki status očekuje različito ponašanje.

U eVrtić sistemu postoji više klasa koje već imaju statusne vrijednosti: Obavijest ima statuse kao što su KREIRANA, POSLANA, NEUSPJELA i PROCITANA, EvidencijaDolaskaOdlaska može biti EVIDENTIRANO ili ODBIJENO, dok SazetakAktivnosti može biti NIJE_GENERISAN, GENERISAN, DJELIMICAN ili PRAZAN. Posebno je korisno primijeniti State patern nad klasom Obavijest, jer ista metoda posalji() ili oznaciKaoProcitanu() ne treba da radi isto kada je obavijest tek kreirana, već poslana ili već pročitana.

Uvođenjem interfejsa IStatusObavijestiState i konkretnih stanja KreiranaState, PoslanaState, NeuspjelaState i ProcitanaState, logika promjene statusa se premješta iz same klase Obavijest u posebne klase. Time se izbjegavaju složeni if/switch blokovi i jasnije se prikazuje dozvoljeni tok obavijesti.

Chain of Responsibility patern

Chain of Responsibility patern koristi se kada zahtjev treba proći kroz više uzastopnih provjera ili obrada, pri čemu svaka klasa u lancu odlučuje da li će zahtjev obraditi, odbiti ili proslijediti sljedećem članu lanca. Na taj način se izbjegava smještanje svih provjera u jednu veliku metodu.

U eVrtić sistemu ovaj patern se može primijeniti kod evidencije dolaska i odlaska djeteta, pristupa osnovnim podacima djeteta i osjetljivih administratorskih akcija. Prije kreiranja zapisa EvidencijaDolaskaOdlaska sistem može prvo provjeriti da li je korisnički nalog aktivan, zatim da

li korisnik ima odgovarajuću ulogu, da li je roditelj povezan s tim djetetom ili odgajatelj s odgovarajućom grupom, da li je QR kod važeći i tek nakon toga kreirati ili odbiti evidenciju.

Predložene klase su IValidacijaHandler, ApstraktniValidacijaHandler, ProvjeraStatusaNalogaHandler, ProvjeraUlogeHandler, ProvjeraPovezanostiDjetetaHandler, ProvjeraQRCodeHandler i KreiranjeEvidencijeHandler. Svaka klasa ima jednu odgovornost i zna samo za sljedeći korak u lancu. Ako se kasnije uvede nova provjera, na primjer provjera radnog vremena vrtića ili dodatna sigurnosna validacija, dodaje se novi handler bez izmjene postojećih handlera.

Template Method patern

Template Method patern koristi se kada više procesa ima isti osnovni redoslijed koraka, ali se pojedini koraci razlikuju u zavisnosti od konkretne podklase. Apstraktna klasa definiše kostur algoritma, dok izvedene klase implementiraju specifične dijelove.

U eVrtić sistemu ovaj patern je pogodan za proces registracije i prijave korisnika. Roditelj i odgajatelj imaju zajedničke korake: unos podataka, validaciju emaila i lozinke, provjeru statusa naloga i dodjelu odgovarajuće uloge. Roditelj ima poseban korak povezivanja sa djetetom preko identifikacionog koda, dok se kod odgajatelja mogu provjeravati podaci o zaposlenju i grupa za koju je zadužen.

AutentifikacijaTemplate bi zato definisao metodu izvršiProces(), a izvedene klase RoditeljRegistracijaProces i OdgajateljRegistracijaProces implementirale bi specifične korake. Na ovaj način je očuvana autentifikacija, ali se izbjegava prebacivanje svih posebnih pravila u jednu veliku klasu.

Observer patern

Observer patern uspostavlja vezu između objekta čije se stanje mijenja i objekata koji trebaju biti obaviješteni o toj promjeni.

U eVrtić sistemu ovaj patern se može koristiti za obavještavanje roditelja. Kada odgajatelj unese ili izmijeni dnevni izvještaj, evidentira dolazak/odlazak, ili kada se generiše sedmični/mjesečni sažetak, roditelj treba dobiti obavijest kroz aplikaciju ili email. Umjesto da svaka klasa ručno poziva slanje obavijesti, DnevniIzvjestajSubject ili SazetakSubject mogu pozvati notify(), a RoditeljObserver i ObavijestService reaguju na promjenu.

Mediator patern

Mediator patern se koristi kada više objekata međusobno komunicira, ali ne želimo da svaki objekat direktno poznaje sve ostale objekte sa kojima sarađuje. Umjesto direktne komunikacije između velikog broja klasa, uvodi se posrednička klasa koja koordinira njihovu komunikaciju.

U eVrtić sistemu ovaj patern se može primijeniti kod procesa evidentiranja dnevnih aktivnosti i obavješćavanja roditelja. Kada odgajatelj unese ili izmijeni dnevni izvještaj, taj događaj može pokrenuti više povezanih akcija: ažuriranje dnevnog zapisa, provjeru evidencije dolaska i odlaska, pripremu podataka za sedmični ili mjesečni sažetak i slanje obavijesti roditelju. Bez Mediator patern, klase kao što su Dnevnilzvjestaj, EvidencijaDolaskaOdlaska, Obavijest i SazetakAktivnosti morale bi direktno pozivati jedna drugu, što bi povećalo međusobnu zavisnost i otežalo održavanje sistema.

Uvođenjem klase EVrticMediator ili AktivnostMediator, komunikacija između ovih objekata se centralizuje. Na primjer, kada se kreira novi dnevni izvještaj, Dnevnilzvjestaj ne mora direktno znati kako se šalje obavijest ili kako se ažurira sažetak. Umjesto toga, obavijesti mediator da se desila promjena, a mediator dalje odlučuje koje akcije treba pokrenuti.

Iterator patern

Iterator patern omogućava prolazak kroz kolekciju podataka bez poznavanja njene interne strukture. Klijent koristi zajednički interfejs za kretanje kroz elemente, dok konkretna kolekcija odlučuje kako se elementi stvarno čuvaju i dohvaćaju.

U eVrtić sistemu Iterator je koristan kod generisanja sažetaka aktivnosti. Sažetak se formira na osnovu većeg broja dnevnih izvještaja za određeno dijete i period. Ti izvještaji mogu biti filtrirani po datumu, djetetu, grupi ili validiranosti. Umjesto da SazetakService direktno obrađuje listu, niz ili rezultat upita iz baze, uvodi se IIzvjestajIterator. KolekcijaDnevnihIzvjestaja kreira DnevnilzvjestajIterator, a SazetakService preko metoda hasNext() i next() prolazi kroz izvještaje.

Command patern

Command patern pretvara korisnički zahtjev u samostalan objekt koji sadrži sve informacije potrebne za izvršenje određene akcije. Time se akcije mogu prosljeđivati, odlagati, stavljati u red čekanja, logovati ili eventualno poništavati.

U eVrtić sistemu ovaj patern je koristan za akcije koje izvršavaju odgajatelj i administrator. Odgajatelj može evidentirati aktivnost, poslati obavijest i pokrenuti generisanje sažetka, dok administrator može dodati, izmijeniti ili deaktivirati profil. Svaka od ovih akcija može biti posebna komanda: EvidentirajAktivnostCommand, PosaljiObavijestCommand, GenerisiSazetakCommand i DeaktivirajProfilCommand.

CommandInvoker prima komandu i poziva execute(), dok konkretna komanda poziva odgovarajući servis ili model. Ovaj pristup je posebno koristan za audit, jer sistem može zapisati ko je izvršio akciju, kada je izvršena i nad kojim objektom. Kod osjetljivih akcija, kao što je deaktivacija profila, moguće je kasnije dodati i undo() logiku.